

A PROJECT REPORT ON

TAMIL VOICE ASSISTANT FOR GOVERNMENT SERVICE INFORMATION

Domain: Speech / NLP

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	30
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	31

CHAPTER I

INTRODUCTION

Government welfare schemes and services such as ration cards, old age pension, scholarships, community and income certificates, and land records are essential for citizens, especially the poor and elderly. Information about eligibility, required documents and procedures is available on many different department websites, mostly in English or in formal written Tamil, and often in long PDF circulars.

Many citizens, particularly elderly people, farmers and people with limited literacy, are not comfortable reading English websites or filling online forms. Typing in Tamil on a phone is also difficult for many. As a result, they depend on middlemen who may charge money or give wrong information, and they make multiple trips to offices because they did not know which documents to bring.

Voice interfaces remove the need to read and type. Advances in speech recognition (such as OpenAI Whisper, which supports Tamil), intent classification and text-to-speech make it possible to build a voice-first assistant. This project, Sevai Kural ('service voice'), answers spoken questions in Tamil about ration card, pension, scholarship and certificate procedures. The user speaks, the system understands the intent and the scheme, retrieves the answer from a curated knowledge base, and replies in both speech and text, including the nearest office and required documents.

LIMITATIONS

1. Many citizens are not comfortable with English websites and forms.
2. Information is scattered across departments.
3. Elderly users find typing difficult.
4. Dependence on middlemen leads to cost and misinformation.

1.1 History of Domain

Speech recognition research began at Bell Labs with the 'Audrey' digit recogniser in 1952. Hidden Markov Models dominated from the 1980s until deep neural networks

improved accuracy dramatically around 2012. Voice assistants such as Siri (2011), Google Now/Assistant and Alexa (2014) brought spoken interaction to consumers, but support for Indian languages came later.

Indian initiatives such as the Bhashini mission (2022) and AI4Bharat's IndicWav2Vec and IndicTrans models have advanced speech and translation technology for Indian languages. OpenAI's Whisper (2022), trained on 680,000 hours of multilingual audio, supports Tamil, making Tamil voice applications practical for students and developers.

1.2 Overview of the Project

The user taps the microphone on the web or mobile page and speaks, for example 'பென்ஷன் வாங்க என்ன ஆவணம் வேண்டும்?' (What documents are needed to get a pension?). The audio is converted to text using Whisper (or Vosk for offline mode). A fine-tuned multilingual BERT/Rasa DIET classifier predicts the intent (eligibility, documents, procedure, office location, status, fees) and an entity extractor identifies the scheme (old age pension, ration card, etc.) and location.

The answer is retrieved from a curated knowledge base prepared from official government sources and reviewed manually. A short Tamil response is generated from templates, converted to speech using gTTS/Coqui TTS, and played back along with the text. The system also provides the nearest office (taluk office, e-Sevai centre) with address and the list of required documents. If the question is not understood, the user is asked to rephrase or is given the helpline number.

1.3 Motivation

Citizens who most need welfare schemes are often the ones least able to access information about them online.

- **Motivation:** To let any citizen get accurate government service information simply by speaking in Tamil.

- **Outcome:** Better awareness of schemes, fewer wasted office visits and reduced dependence on middlemen.

1.4 Aim of the Project

The project is entitled as “**TAMIL VOICE ASSISTANT FOR GOVERNMENT SERVICE INFORMATION**”. The aim of this project is to develop a Tamil voice assistant that understands spoken questions about government services, retrieves accurate information from a curated knowledge base, and responds in spoken and written Tamil along with nearby office details and required documents.

1.5 Objectives

- To convert Tamil speech to text using Whisper/Vosk.
- To classify user intents and extract scheme entities.
- To build a curated, verified knowledge base of schemes and procedures.
- To generate short Tamil responses and convert them to speech.
- To provide office locations and document checklists.

1.6 Literature Review

Radford et al. (2023): Presented Whisper, a speech recognition model trained with large-scale weak supervision on 680,000 hours of multilingual audio, which approaches human-level robustness and supports Tamil.

Bunk et al. (2020): Introduced DIET (Dual Intent and Entity Transformer), a lightweight architecture for joint intent classification and entity recognition used in Rasa.

Kakwani et al. (2020): Released IndicNLPSuite with IndicCorp monolingual corpora and IndicBERT, a multilingual ALBERT model for 12 Indian languages including Tamil.

Javed et al. (2022): Developed IndicWav2Vec and benchmarks for automatic speech recognition in Indian languages, showing the benefit of pre-training on Indian language audio.

Bocklisch et al. (2017): Described Rasa, an open-source language understanding and dialogue management framework for building conversational assistants.

Government of Tamil Nadu e-Sevai: The e-Sevai portal lists services such as community, income and nativity certificates with required documents and fees, which serve as sources for the knowledge base.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

Currently citizens get information by visiting offices, asking village administrative officers, calling helplines that may be busy, reading department websites, or depending on agents. Some chatbots exist on websites but are text-based and English-centric.

DISADVANTAGES

- Information scattered across many websites.
- English and text-heavy interfaces.
- Long waiting time on helplines.
- Middlemen charge fees and may mislead.
- Repeated office visits due to missing documents.

2.2 Proposed System

The proposed Sevai Kural assistant provides a voice-first, Tamil interface. Speech recognition, intent classification and entity extraction understand the question; a curated knowledge base gives verified answers; and text-to-speech replies in Tamil. Office locations and document checklists reduce repeated visits.

ADVANTAGES

- No need to read or type; speak in Tamil.
- Single place for many schemes.
- Verified answers from official sources.
- Document checklist and nearest office.
- Available 24x7.

2.3 Project Schedule

The development of Sevai Kural follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the Sevai Kural system are listed below.

- **Speech capture:** Record the user's question from the microphone.
- **Speech to text:** Convert Tamil speech to text.
- **Intent detection:** Classify intent: eligibility, documents, procedure, office, fees, status.
- **Entity extraction:** Identify scheme name and district.
- **Answer retrieval:** Fetch answer from the knowledge base.
- **Response generation:** Generate a short Tamil response.
- **Text to speech:** Play the response as Tamil audio.
- **Office finder:** Show nearest office with address and timing.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Accuracy:** Intent accuracy above 90% on test utterances.
- **Latency:** Complete response within 4 seconds.

- **Reliability:** Fallback to helpline when confidence is low.
- **Usability:** Large microphone button, slow clear speech output.
- **Maintainability:** Admin can update scheme information without code changes.
- **Privacy:** Audio deleted after transcription.

3.3 User Requirements

- Speak in natural Tamil and be understood
- Hear the answer in simple Tamil
- Know which documents to bring
- Know where the nearest office is
- Replay or read the answer

3.4 System Requirements

- FastAPI server with Whisper, intent model and TTS
- SQLite knowledge base
- React web front end with microphone access
- Internet connection (offline Vosk mode optional)

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System	: Intel i5 Processor 2.4 GHz or above
Hard Disk	: 256 GB SSD
RAM	: 16 GB
GPU	: Optional NVIDIA GPU for faster Whisper
Microphone and speaker	: Smartphone or headset
Monitor	: 15" colour monitor

4.2 Software Specification

Operating System	: Windows 10 / Ubuntu 22.04
Coding Language	: Python 3.10, JavaScript
Speech to Text	: OpenAI Whisper (small) / Vosk Tamil model
NLU	: Rasa DIET or fine-tuned IndicBERT
Text to Speech	: gTTS / Coqui TTS
Back End	: FastAPI
Front End	: React JS
Database	: SQLite

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed Sevai Kural system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Whisper and Vosk provide Tamil speech recognition; Rasa and HuggingFace provide intent classification; gTTS provides Tamil speech output. All run on a standard server. The project is technically feasible.

ECONOMIC FEASIBILITY

All tools are open source or free. The system can be hosted at an e-Sevai centre or on a low-cost server. It is economically feasible.

OPERATIONAL FEASIBILITY

Users only press a button and speak. Admin staff update the knowledge base through forms. It is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI),

which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

OPENAI WHISPER / VOSK

Whisper is an open-source automatic speech recognition (ASR) model trained on 680,000 hours of multilingual audio. It supports Tamil and many other languages and is robust to accents and background noise.

Vosk is an offline speech recognition toolkit with small models that run on low-end devices. Either engine converts the user's speech into text for the intent classifier.

RASA

Rasa is an open-source machine learning framework for building text and voice based conversational assistants. Rasa NLU performs intent classification and entity extraction, and Rasa Core manages the dialogue.

Its DIET classifier can be trained on a few hundred example sentences per intent, including Tamil sentences.

HUGGINGFACE TRANSFORMERS

HuggingFace Transformers is an open-source library that provides thousands of pre-trained transformer models for tasks such as text classification, named entity recognition, question answering and translation, in more than 100 languages.

Multilingual models such as mBERT, XLM-RoBERTa and IndicBERT understand Tamil and English, so a single model can be fine-tuned to classify text in both languages.

- Pre-trained multilingual models

- Simple pipeline() API
- Works with both PyTorch and TensorFlow

TEXT-TO-SPEECH (GTTS / COQUI TTS)

gTTS (Google Text-to-Speech) is a Python library that converts text into spoken audio using Google's TTS service and supports Tamil. Coqui TTS is an open-source deep learning toolkit for offline speech synthesis.

FASTAPI

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. It is built on Starlette for the web parts and Pydantic for data validation.

FastAPI automatically generates interactive API documentation (Swagger UI and ReDoc), validates request bodies, and supports asynchronous request handling, which gives performance comparable to Node.js and Go. It is well suited for serving machine learning models with low latency.

- Automatic request validation with Pydantic
- Auto-generated OpenAPI documentation
- Asynchronous and very fast

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI

- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

SQLITE

SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured SQL database engine. Unlike client-server databases, SQLite reads and writes directly to ordinary disk files.

Because it requires no separate server process and no configuration, SQLite is ideal for prototypes, mobile applications and small deployments.

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of Sevai Kural is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

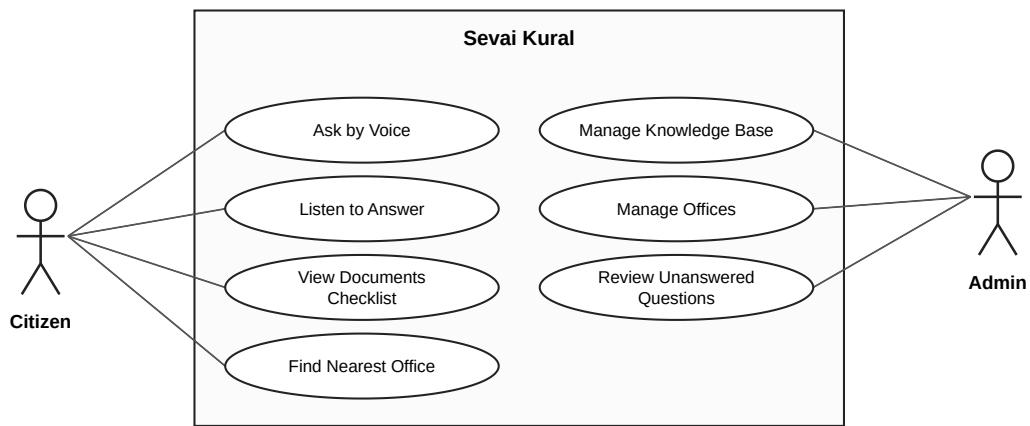


Fig 1: Use case diagram of Sevai Kural

6.2 System Architecture

Sevai Kural follows a speech pipeline architecture: speech recognition, natural language understanding, knowledge retrieval, response generation and speech synthesis, exposed through a FastAPI service to a React client.

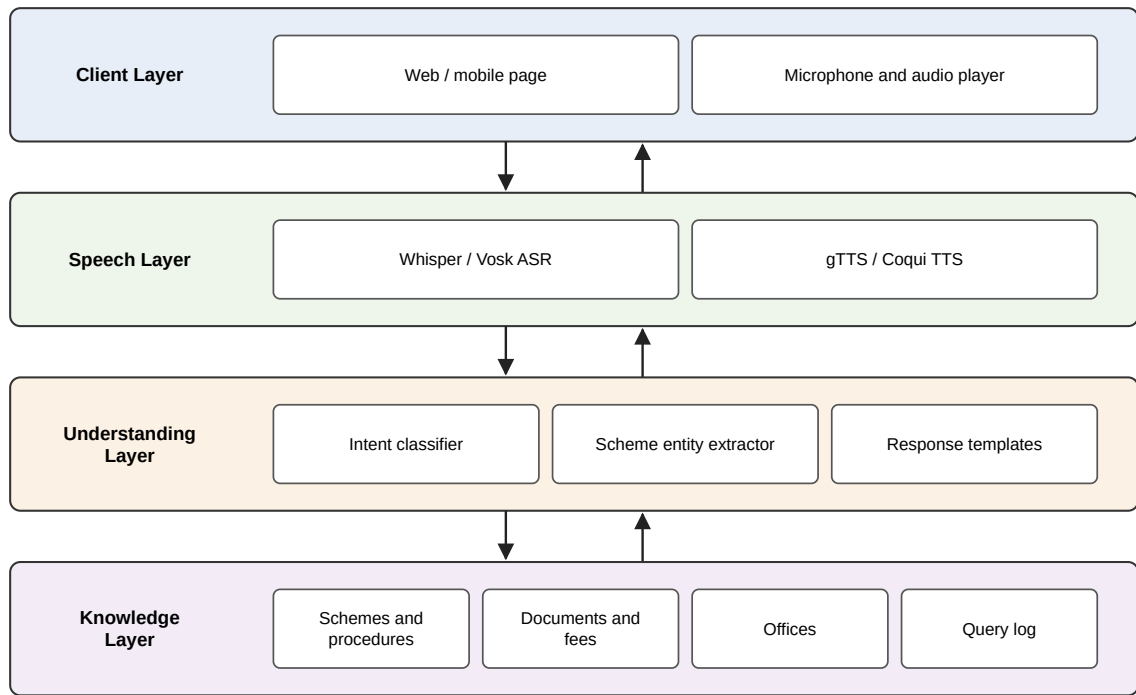


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

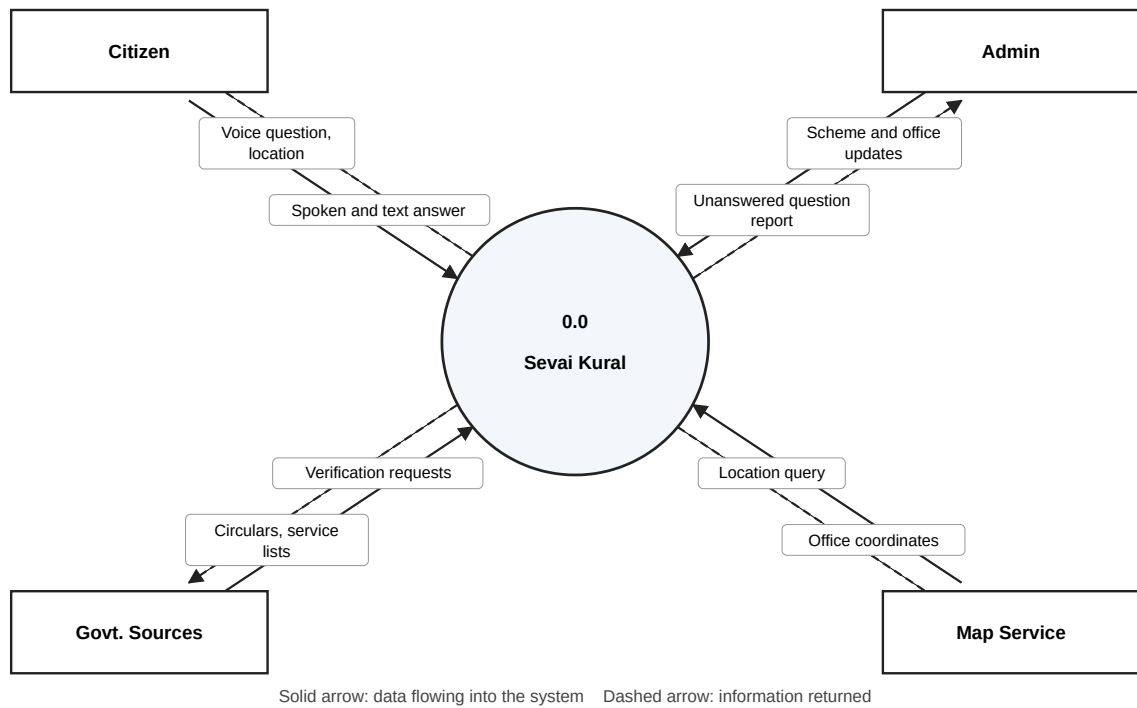


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

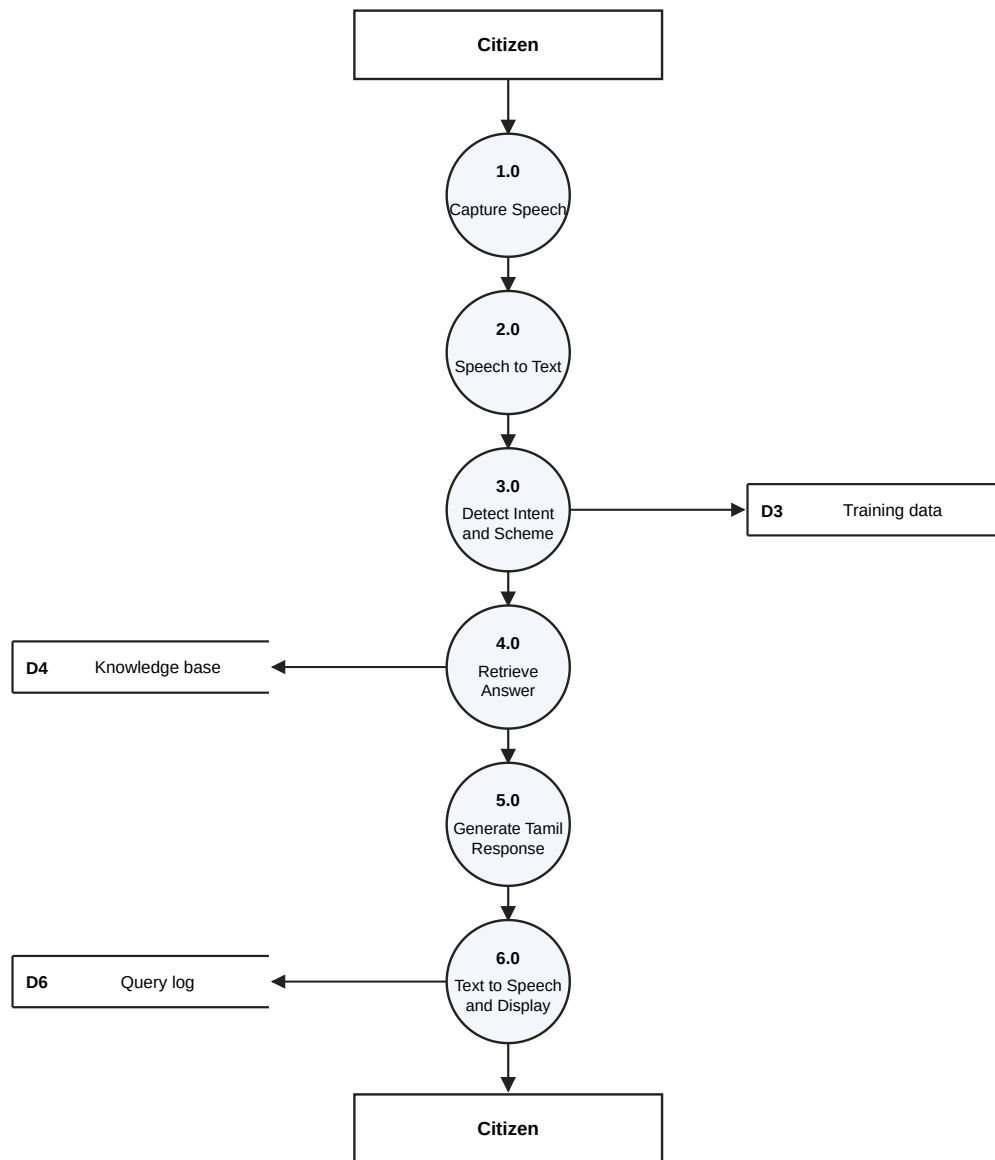


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

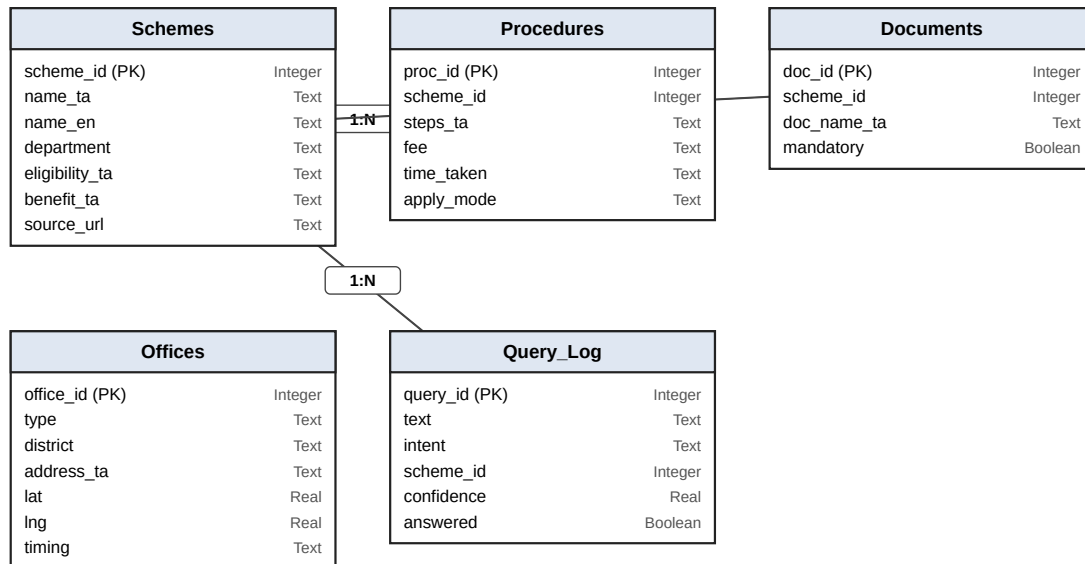


Fig 5: Entity relationship diagram

6.5 Table Design

SCHEMES

FIELD NAME	DATA TYPE	DESCRIPTION
scheme_id	Integer	Primary key
name_ta	Text	Scheme name in Tamil
name_en	Text	Scheme name in English
department	Text	Department
eligibility_ta	Text	Eligibility in Tamil
benefit_ta	Text	Benefit in Tamil
source_url	Text	Official source

PROCEDURES

FIELD NAME	DATA TYPE	DESCRIPTION
proc_id	Integer	Primary key
scheme_id	Integer	Foreign key to Schemes
steps_ta	Text	Steps in Tamil
fee	Text	Fee
time_taken	Text	Processing time

apply_mode	Text	online / e-Sevai / office
------------	------	---------------------------

DOCUMENTS

FIELD NAME	DATA TYPE	DESCRIPTION
doc_id	Integer	Primary key
scheme_id	Integer	Foreign key to Schemes
doc_name_ta	Text	Document name in Tamil
mandatory	Boolean	Mandatory or optional

OFFICES

FIELD NAME	DATA TYPE	DESCRIPTION
office_id	Integer	Primary key
type	Text	e-Sevai / taluk / VAO
district	Text	District
address_ta	Text	Address
lat	Real	Latitude
lng	Real	Longitude
timing	Text	Working hours

QUERY_LOG

FIELD NAME	DATA TYPE	DESCRIPTION
query_id	Integer	Primary key
text	Text	Transcribed question
intent	Text	Predicted intent
scheme_id	Integer	Detected scheme
confidence	Real	Confidence
answered	Boolean	Answered or fallback

6.6 Algorithm Design

Each spoken question goes through the following pipeline.

1. **Capture speech** – The browser records audio with the MediaRecorder API and sends it to the server.
2. **Convert speech to text (Whisper / Vosk Tamil)** – Whisper small model with language='ta'; Vosk is used offline.
3. **Detect intent using a classifier** – Intents: eligibility, documents, procedure, fees, office_location, application_status, greeting, out_of_scope.
4. **Extract entities such as scheme name** – Fuzzy matching against Tamil and English scheme names and synonyms (e.g., 'பென்ஷன்', 'முதியோர் உதவித்தொகை').
5. **Retrieve answer from curated knowledge base** – SQL query based on intent and scheme.
6. **Generate short Tamil response** – Response templates keep answers short and clear (2-3 sentences) with the main facts.
7. **Convert text to speech** – gTTS (lang='ta') generates MP3 audio.
8. **Offer nearest office details and required documents** – Haversine distance finds the nearest office from the user's location or district.

PSEUDO CODE

```
ON AUDIO(a, location):
  text <- WHISPER.transcribe(a, language="ta")
  intent, conf <- INTENT_MODEL(text)
  scheme <- FUZZY_MATCH(text, scheme_names_and_synonyms)
  IF conf < 0.6 OR (intent needs scheme AND scheme is None):
    reply <- "மன்னிக்கவும், மீண்டும் சொல்லுங்கள். உதவி எண்: 1100"
  ELSE:
    facts <- KB.lookup(intent, scheme)
    reply <- TEMPLATE[intent].fill(facts)
    IF intent IN {procedure, documents, office_location}:
      office <- NEAREST(offices, location)
  audio <- TTS(reply, "ta"); LOG(text, intent, scheme, conf)
  RETURN text, reply, audio, documents, office
```

6.7 Modules

The modules are divided between the Citizen and the Admin users.

MODULES

CITIZEN

1. ASK BY VOICE
2. LISTEN TO ANSWER
3. VIEW DOCUMENTS CHECKLIST
4. FIND NEAREST OFFICE

ADMIN

1. MANAGE KNOWLEDGE BASE
2. MANAGE OFFICES
3. REVIEW UNANSWERED QUESTIONS

CITIZEN

ASK BY VOICE

The citizen presses the microphone button and asks a question in Tamil. Text input is also available.

LISTEN TO ANSWER

The assistant replies in spoken Tamil with the same text shown on screen; the user can replay it.

VIEW DOCUMENTS CHECKLIST

For procedure questions, a checklist of required documents and fees is shown.

FIND NEAREST OFFICE

The citizen shares location or district and receives the nearest e-Sevai centre or taluk office with timings and map link.

ADMIN

MANAGE KNOWLEDGE BASE

Admin adds and updates schemes, eligibility, documents, fees and procedures from official circulars.

MANAGE OFFICES

Admin updates office addresses, timings and contact numbers.

REVIEW UNANSWERED QUESTIONS

Low-confidence questions are listed so that new training examples and answers can be added.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

Sevai Kural was implemented with a FastAPI back end. Speech recognition uses the open-source Whisper 'small' model with the Tamil language option; for offline kiosks the Vosk Tamil model was tested. The intent classifier was trained with Rasa NLU (DIET) on about 1,600 Tamil and Tanglish example utterances across 8 intents. Responses are converted to speech with gTTS.

7.2 Implementation

The knowledge base was manually prepared from official Tamil Nadu government department pages and e-Sevai service lists for 25 commonly used services and verified twice. Each answer records its source URL and last-verified date so that information can be kept current.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	Speech to text	Spoken 'ரேஷன் கார்டு எப்படி வாங்குவது'	Correct Tamil transcription	Pass
TC02	Intent - documents	'பென்ஷனுக்கு என்ன ஆவணம்'	documents + old age pension	Pass
TC03	Intent - eligibility	'scholarship யாருக்கு கிடைக்கும்'	eligibility + scholarship	Pass
TC04	Office finder	Location Thanjavur	Nearest e-Sevai centre with address	Pass
TC05	Out of scope	'இன்று கிரிக்கெட் ஸ்கோர் என்ன'	Polite out-of-scope reply	Pass
TC06	Low confidence	Unclear noisy audio	Ask to repeat + helpline	Pass
TC07	TTS output	Any answer	Clear Tamil audio plays	Pass
TC08	Admin update	Change fee of certificate	New fee in next answer	Pass

PERFORMANCE EVALUATION

Evaluation on 400 test utterances from 20 speakers:

Component	Metric	Value
Whisper small (Tamil)	Word error rate	24.6%
Intent classifier	Accuracy	92.3%
Scheme entity	Accuracy	94.0%
End-to-end answer correct	Accuracy	86.5%
Response time	Average	3.1 s

7.4 Source Code

assistant.py

```
import math, sqlite3, uuid, whisper
from fastapi import FastAPI, UploadFile, Form
from rapidfuzz import process, fuzz
from rasa.core.agent import Agent
from gtts import gTTS

app = FastAPI()
asr = whisper.load_model("small")
nlu = Agent.load("models/sevai_nlu.tar.gz")
db = sqlite3.connect("kb.sqlite", check_same_thread=False); db.row_factory =
sqlite3.Row
SYN = {r["syn"]: r["scheme_id"] for r in db.execute("SELECT syn, scheme_id FROM
scheme_synonyms")}
FALLBACK = "மன்னிக்கவும், உங்கள் கேள்வி புரியவில்லை. மீண்டும் சொல்லுங்கள் அல்லது 1100 என்ற
எண்ணை அழைக்கவும்."

TEMPLATES = {
    "eligibility": "{name} பெற தகுதி: {eligibility}",
    "documents": "{name} விண்ணப்பிக்க தேவையான ஆவணங்கள்: {docs}.",
    "procedure": "{name}: {steps} கட்டணம் {fee}. சுமார் {time} ஆகும்.",
    "fees": "{name} கட்டணம் {fee}.",
}

def find_scheme(text):
    m = process.extractOne(text, SYN.keys(), scorer=fuzz.partial_ratio)
    return SYN[m[0]] if m and m[1] >= 80 else None

def nearest_office(lat, lng):
    rows = db.execute("SELECT * FROM offices WHERE type='e-Sevai').fetchall()
    d = lambda r: math.hypot(r["lat"] - lat, (r["lng"] - lng) *
math.cos(math.radians(lat)))
    return dict(min(rows, key=d)) if rows else None

@app.post("/ask")
async def ask(audio: UploadFile, lat: float = Form(10.78), lng: float =
Form(79.13)):
    path = f"tmp/{uuid.uuid4().hex}.webm"
    open(path, "wb").write(await audio.read())
    text = asr.transcribe(path, language="ta")["text"].strip()
    parsed = await nlu.parse_message(text)
    intent, conf = parsed["intent"]["name"], parsed["intent"]["confidence"]
    sid = find_scheme(text)
    docs, office = [], None
    if conf < 0.6 or intent not in TEMPLATES or sid is None:
        reply = FALLBACK if intent != "out_of_scope" else "நான் அரசு சேவை
தகவல்களுக்கு மட்டுமே உதவ முடியும்."
    else:
        s = db.execute("SELECT * FROM schemes s JOIN procedures p
USING(scheme_id) WHERE scheme_id=?", (sid,)).fetchone()
        docs = [r["doc_name_ta"] for r in db.execute("SELECT doc_name_ta FROM
documents WHERE scheme_id=?", (sid,))]
        reply = TEMPLATES[intent].format(name=s["name_ta"],
eligibility=s["eligibility_ta"], docs=", ".join(docs),
steps=s["steps_ta"], fee=s["fee"],
time=s["time_taken"])
        office = nearest_office(lat, lng)
```

```

mp3 = f"static/{uuid.uuid4().hex}.mp3"
gTTS(reply, lang="ta", slow=False).save(mp3)
db.execute("INSERT INTO query_log(text, intent, scheme_id, confidence,
answered) VALUES (?, ?, ?, ?, ?)",
            (text, intent, sid, conf, reply != FALLBACK)); db.commit()
return {"question": text, "intent": intent, "reply": reply, "audio": "/" +
mp3, "documents": docs, "office": office}

```

data/nlu.yml (Rasa training data - excerpt)

```

version: "3.1"
nlu:
- intent: documents
  examples: |
    - [பென்ஷன்](scheme) வாங்க என்ன ஆவணம் வேண்டும்
    - [ரேஷன் கார்டு](scheme) க்கு என்ன documents கொடுக்கணும்
    - [வருமான சான்றிதழ்](scheme) apply பண்ண என்ன தேவை
- intent: eligibility
  examples: |
    - [முதியோர் உதவித்தொகை](scheme) யாருக்கு கிடைக்கும்
    - [scholarship](scheme) க்கு தகுதி என்ன
- intent: procedure
  examples: |
    - [சாதி சான்றிதழ்](scheme) எப்படி வாங்குவது
    - [ரேஷன் கார்டு](scheme) online ல் எப்படி apply பண்ணுது
- intent: office_location
  examples: |
    - பக்கத்துல இ-சேவை மையம் எங்கே இருக்கு
    - தாலுகா அலுவலகம் எங்கே

```

7.5 Screen Layout

SCREENSHOTS:

VOICE ASSISTANT

Sevai Kural

AskSchemesOfficesHistoryAdmin

சேவை குரல் - உங்கள் கேள்வியை கேளுங்கள்

முதியோர் உதவித்தொகை வாங்க என்ன ஆவணம் வேண்டும்?

முதியோர் உதவித்தொகை விண்ணப்பிக்க தேவையான ஆவணங்கள்: ஆதார் அட்டை, வயது சான்று, வருமான சான்றிதழ், வங்கி கணக்கு புத்தகம், புகைப்படம்.

பக்கத்துல எங்கே apply பண்ணலாம்?

அருகிலுள்ள இ-சேவை மையம்: வட்டாட்சியர் அலுவலகம், தஞ்சாவூர் (1.8 கி.மீ). நேரம் காலை 10 முதல் மாலை 5 வரை.

மைக் பொத்தானை அழுத்தி பேசுங்கள்...

DOCUMENT CHECKLIST

Sevai Kural

AskSchemesOfficesHistoryAdmin

Old Age Pension - document checklist

முதியோர் உதவித்தொகை (OAP)

Eligibility	Age 60+, destitute / annual income below limit
Documents	Aadhaar, age proof, income certificate, bank passbook, photo
Apply at	e-Sevai centre / Taluk office
Fee	Free
Time taken	About 30 days
Source	Social Welfare Dept. (verified 01-10-2026)

Play in Tamil

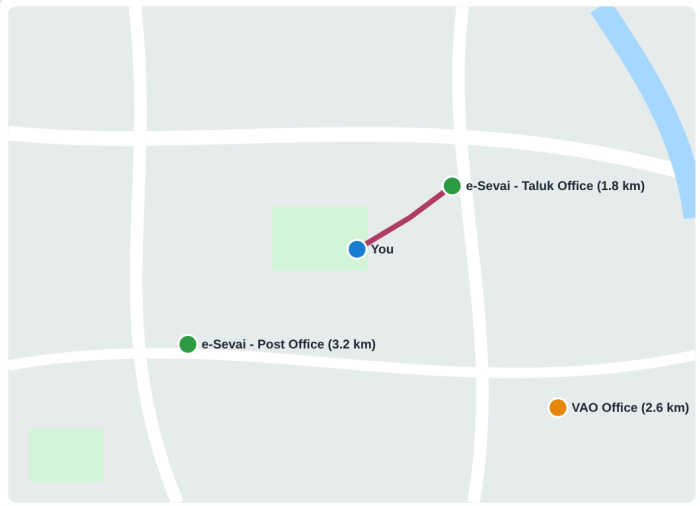
Print / share this checklist

NEAREST OFFICE

Sevai Kural

AskSchemesOfficesHistoryAdmin

Nearest e-Sevai centres



Nearest	Taluk Office e-Sevai
Distance	1.8 km
Timing	10:00 AM - 5:00 PM
Closed	Sundays, govt holidays
Phone	04362-2xxxxx

ADMIN - KNOWLEDGE BASE

Sevai Kural

AskSchemesOfficesHistoryAdmin

Knowledge base - schemes

Department: AllNeeds review: Yes

Scheme	Department	Last verified	Queries (30d)	Status
Ration card (new)	Civil Supplies	02-10-2026	1,420	VERIFIED
Old age pension	Social Welfare	01-10-2026	980	VERIFIED
Post-matric scholarship	Adi Dravidar Welfare	12-06-2026	640	REVIEW
Income certificate	Revenue	28-09-2026	1,105	VERIFIED

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Tamil Voice Assistant for Government Service Information allows citizens to obtain accurate information about welfare schemes and certificates simply by speaking in Tamil. Using Whisper for speech recognition, a DIET intent classifier and a curated knowledge base, the system answered 86.5% of test questions correctly end-to-end with an average response time of about 3 seconds.

Document checklists and nearest office information reduce repeated visits and dependence on middlemen. The project demonstrates how speech and language technology can make public services more inclusive for elderly and low-literacy citizens.

8.2 Future Enhancement

- Application status tracking through department APIs.
- Support for regional dialects and other Indian languages via Bhashini.
- IVR telephone access for users without smartphones.
- Retrieval-augmented generation over official circulars with citation.
- Kiosk deployment at bus stands and panchayat offices.

CHAPTER IX

BIBLIOGRAPHY

1. Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I., "Robust Speech Recognition via Large-Scale Weak Supervision", Proc. ICML, 2023.
2. Bunk, T., Varshneya, D., Vlasov, V., and Nichol, A., "DIET: Lightweight Language Understanding for Dialogue Systems", arXiv:2004.09936, 2020.
3. Kakwani, D. et al., "IndicNLP Suite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages", Findings of EMNLP, 2020.
4. Javed, T. et al., "Towards Building ASR Systems for the Next Billion Users", Proc. AAAI, 2022.
5. Bocklisch, T., Faulkner, J., Pawlowski, N., and Nichol, A., "Rasa: Open Source Language Understanding and Dialogue Management", arXiv:1712.05181, 2017.
6. Jurafsky, D., and Martin, J. H., Speech and Language Processing, 3rd Edition draft, Stanford University.

WEBSITE REFERENCE

1. <https://github.com/openai/whisper>
2. <https://rasa.com/docs>
3. <https://www.tnesevai.tn.gov.in>
4. <https://bhashini.gov.in>